

ONE-PASS: Single Forward Pass Decoding for Listwise Reranking

Yingpeng Du

Nanyang Technological University
Singapore
yingpeng.du@ntu.edu.sg

Tianjun Wei

Nanyang Technological University
Singapore
tjwei2-c@my.cityu.edu.hk

Zhu Sun*

Singapore University of Technology and Design
Singapore
zhu_sun@sutd.edu.sg

Jie Zhang

Nanyang Technological University
Singapore
zhangj@ntu.edu.sg

Abstract

Large Language Models (LLMs) have been widely adopted in ranking systems, specifically for reranking tasks. Despite the effectiveness, the auto-regressive decoding of LLMs leads to inference latency due to the memory-bandwidth-bound. To alleviate this bottleneck, prior studies have explored single token decoding as an approximation, but they suffer from performance degradation at the tail positions of ranking. In this paper, we propose a Single Forward Pass (SFP)-based method to pre-verify multiple rankings using tree attention, approximating auto-regressive decoding by relevant sub-rankings at each step. However, verifying all possible ranking permutations will lead to factorial-level token computation ($N!$), making it intractable within SFP. To this end, we first reduce item ranking permutations to combinations (2^N), based on the empirical observation that the LLM's next-item generation is less sensitive to the exact ordering of preceding items. Furthermore, we divide the full ranking into K sub-rankings and aggregate their individual probabilities, which further decreases the verification space to $2^{\lceil N/K \rceil} \cdot K \ll 2^N$. However, naïvely aggregating sub-ranking probabilities leads to inaccurate estimates in listwise ranking. To overcome this, we introduce a Möbius inversion model that explicitly decompose the individual contribution of subsets within a complete lattice, as verified by tree attention. Then, we learn their higher-order effects with a hierarchical self-attention model to reconstruct the full ranking probability. Experiments on both information retrieval and recommendation tasks show the effectiveness of our proposed method.

CCS Concepts

• Information systems → Information systems applications.

Keywords

Ranking task, Large language model, Möbius inversion.

ACM Reference Format:

Yingpeng Du, Zhu Sun, Tianjun Wei, and Jie Zhang. 2026. ONE-PASS: Single Forward Pass Decoding for Listwise Reranking. In *Proceedings of the*

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792243>

ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792243>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18297818>.

1 Introduction

Large Language Models (LLMs) [37] have recently achieved remarkable success in ranking systems, including information retrieval (IR) and recommender systems (RS), especially for reranking tasks [7, 22, 27, 38]. Given a query (e.g., a user profile in RSs), ranking systems aim to order a set of candidate items according to their relevance or preference [6]. The integration of LLMs has significantly improved ranking effectiveness, but their practical deployment is hindered by the high inference latency of *auto-regressive decoding*. As shown in Figure 1 (A), each decoding round necessitates loading the billion-parameter LLM from high-bandwidth memory (HBM) into the on-chip cache of modern accelerators (e.g., GPUs), resulting in inefficient and time-consuming generation.

To this end, speculative decoding (SD) accelerates text generation by parallel verification of multiple tokens [16], reducing the decoding round for efficient inference of LLMs. While effective in preserving position-wise accuracy, SD relies on the multi-round decoding, still bounded by the HBM and making the rounds of verification agnostic, hindering the latency guarantees required by ranking systems in real-world applications. To this end, prior studies investigate the Single Token Decoding (STD) strategy for ranking tasks, where the ranking score of each candidate is approximated using only the logits of the first generated token from a single forward pass [22–24], as shown in Figure 1 (B). However, these methods, such as FIRST [24], can approximate top-ranked items well, but they suffer from severe degradation in the tail rankings (as shown in Figure 2 (B)).

Inspired by the tree attention mechanism [2], it enables the processing of multiple rankings in parallel within Single Forward-Pass (SFP) for LLMs, where the encoding of a certain node relies solely on its parent nodes. As shown in Figure 1 (C), the common prefix of *trie* can pre-verify several sub-rankings, demonstrating its potential for conducting auto-regressive decoding by verified rankings within an SFP. Specifically, suppose we have N candidate items $\mathcal{D}$ for reranking, for t -th step in auto-regressive decoding, we need to select the item with maximum possibility based on

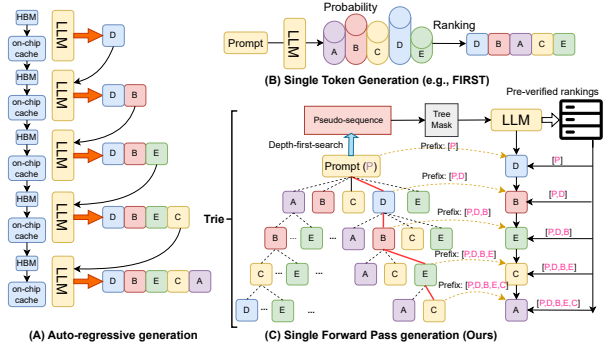


Figure 1: Different schemes of decoding for reranking tasks. (A) Auto-regressive decoding shows high communication between HBM and on-chip; (B) Single token decoding shows low accuracy; (C) Our single forward pass generation shows low communication and high accuracy, showing potential for LLM-based reranking.

the forward-passing $t - 1$ preceding items by LLMs. Therefore, it requires pre-verifying (forward-pass) all permutations of subsets of these items by LLMs, i.e., $\cup_{c \subseteq \mathcal{D}} \text{Perm}(c)$. Although the trie encoding can estimate all these permutations by encoding $N!$ tokens with common prefixes, it is intractable to encode these extremely large-scale tokens for LLMs within an SFP.

In this paper, we present two reductions to approximate auto-regressive decoding by pre-verifying multiple sub-rankings, making SFP tractable and effective ($N! \rightarrow 2^N \rightarrow K \cdot 2^{\lceil N/K \rceil}$) as shown in Table 1. **(i) Permutations $\rightarrow$ Combinations.** Our empirical finding shows that the LLM’s next-item generation is not extremely sensitive to the ordering of already ranked items, where detailed analysis can be found in Section 4.1. To this end, we estimate $\mathbb{P}(i|I_k)$ conditioned on an *unordered* set $S_k = \text{set}(I_k)$, where $\mathbb{P}(\cdot)$ denotes the probability of the next item selection in LLMs and I_k denotes the ranking of preceding items. Therefore, it can collapse permutation families ($N!$) into a less computationally intensive combination of families (2^N). **(ii) Full ranking $\rightarrow$ sub-rankings.** To further reduce the computational cost, we partition the candidate set $\mathcal{D}$ into K disjoint subsets $\mathcal{D}_1, \dots, \mathcal{D}_K$ and verify them in parallel, shrinking the space from 2^N to $K \cdot 2^{\lceil N/K \rceil} \ll 2^N$. Although such a strategy can drastically reduce the computational cost, naively aggregating sub-ranking probabilities may lead to inaccurate estimation for listwise probability, i.e., $\mathbb{P}(i|\cup_k S_k) \neq \sum_{k=1}^K \mathbb{P}(i|S_k)$ where $S_k \subseteq \mathcal{D}_k$, as it ignores cross-subset contribution to full ranking. To tackle this problem, we introduce a Möbius inversion model [4] that learns from dual sub-rankings containing partial cross-subset information, modeling the overall cross-subset contributions for full ranking approximation [1]. Specifically, we first encode dual sub-rankings ($\mathcal{D}'_1, \dots, \mathcal{D}'_K$) containing partial cross-subset information (e.g., $\mathcal{D}'_i \cap \mathcal{D}_j \neq \emptyset$ and $\mathcal{D}'_i \cap \mathcal{D}_k \neq \emptyset$ with some i, j, k) w.r.t. the existing sub-rankings ($\mathcal{D}_1, \dots, \mathcal{D}_K$). Then, the availability of complete Boolean lattices of combinations ($\subseteq, 2^{\mathcal{D}_i}$) makes it possible to decouple listwise probability into individual effects. To this end, we employ the Möbius inversion to bridge the overall listwise probability and the individual effects through reverse-engineering, which can be viewed as an encoder-decoder framework. For the encoding phase, the Möbius transform on the Boolean lattice explores

Table 1: Required #tokens for SFP among different strategies. Permutation ($N!$) vs. Combination (2^N) vs. Sub-rankings ($K \cdot 2^{\lceil N/K \rceil}$)

N (item size)	1	5	10	15	20	25
Permutation	1	120	3.60e+06	1.30e+12	2.40e+18	1.60e+25
Combination	1	32	1024	32768	1048576	3.40e+07
sub-rankings	K=2	1	16	64	512	2048
	K=3	1	12	48	96	384
	K=4	1	16	32	64	128
	K=5	1	10	20	40	80

the unique individual effects of different subsets. For the decoding phase, we adopt the hierarchical self-attention model for the inverse transform, recomposing to recover the listwise probability.

In summary, we propose an SFP-based method, named ONE-PASS, approximating auto-regressive decoding by pre-verifying multiple sub-rankings with the tree attention. To alleviate the intensive tokens within SFP, we reduce item ranking permutations to combinations and divide the full ranking into sub-rankings, making it efficient for listwise reranking. To alleviate accuracy degradation, we employ the Möbius inversion to decouple into individual effects of each subset, approximating the probability of full ranking with a hierarchical self-attention model. Experiments on both IR and RS tasks show the effectiveness of our proposed method.

2 Related Work

Typically, ranking systems commonly employ a multi-stage pipeline for serving: first selecting several candidates from the large-scale item corpus [3], and then ranking them by dedicated models. Recently, LLMs have been widely used in ranking systems such as IR systems [22, 23, 38] and RSs [13, 27–29] for reranking.

Despite effectiveness, the nature of auto-regressive decoding in LLMs makes LLM-based ranking come at the expense of increased latency [8, 19, 21, 24]. Among these methods, FIRST [24] is the effective method to speed up the LLM decoding for ranking tasks. Specifically, it first employs a learning-to-rank loss to fine-tune the target LLMs, and then leverages the output logits of the first generated identifier to directly produce a ranked ordering of the candidates. Therefore, such a reranking process can be completed within SFP, largely reducing the communication between HBM and on-chip. However, these methods suffer from severe degradation in tail position due to they only rely on the logit of the first generated token, leaving a huge gap between the first token and the tail tokens.

Besides these methods, speculative decoding can accelerate the decoding of LLMs by drafting and verifying multi-token sequences based on the LLM encoding, ensuring output consistency with auto-regressive decoding [9, 16, 26, 32, 33]. Specifically, they speculate multiple potential future tokens (i.e., draft phase [2, 5, 10, 12, 25, 26]) and then subsequently confirm the correctness of these tokens within a one-time LLM forward pass (i.e., verification phase [10, 12, 14, 25, 26, 32, 35, 36]). We notice the work of [31] employs tree attention for speculative decoding in RS tasks, but its focus on accelerating the chain-of-thought process [18, 30], rather than reranking, makes its task distinct from ours. In addition, these speculative decoding methods also rely on the multi-round decoding bounded by the communication between HBM and on-chip cache, and making the latency in real-world applications.

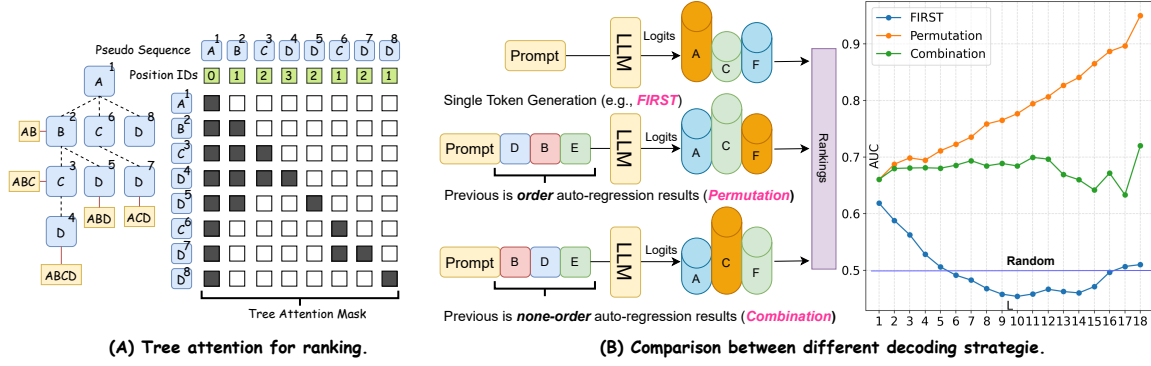


Figure 2: (A) Tree attention for parallel ranking verification. (B) Comparison between different decoding strategies.

3 Preliminary

This section first introduces the background of the auto-regressive decoding and the tree mask for decoding. Then, we formulate the problem of ranking systems.

Auto-regressive decoding in LLMs. LLMs take discrete token sequences as input, which is denoted as $x_{1:t} \in \mathcal{T}^t$ of length t , where $\mathcal{T}$ denotes the token corpus. The output of LLMs characterizes the probability distribution of the next token based on input tokens. Specifically, the probability of the t -th token is determined by all previous (including generated) tokens, represented as $\mathbb{P}(y|x_{1:t-1}) \in \mathbb{R}^{\mathcal{T}}$. Then, the next input token o_t is obtained by sampling from $\mathbb{P}(y|x_{1:t-1})$ using different methods such as greedy, top- K sampling [15]. Finally, the generated token o_t is added to the input $x_{1:t} \leftarrow x_{1:t-1} + \{o_t\}$, and LLM will iteratively generate tokens until meeting the token [EOS]. We define x^0 as the query or prompt tokens. In this paper, we adopt the greedy auto-regressive decoding strategy for reproducibility, which can be formulated as $o_t = \arg\max \text{LLM}(y|x^0, x_{1:t-1})$.

Tree attention. Tree attention [2] can validate multiple potential draft sequences in parallel, as illustrated in Figure 2 (A). Firstly, we construct a token tree based on several draft sequences with shared common preceding nodes. Then, we adopt a depth-first search algorithm to represent this tree into a pseudo-sequence $\tilde{x} = (\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_\ell)$, where ℓ denotes the total number of nodes in this tree. Finally, we adjust the attention mask $\mathbf{M} \in \{0, 1\}^{\ell \times \ell}$ and position IDs $\mathbf{P} \in \mathbb{N}^\ell$ so that each tree node $\tilde{x}_j$ can only attend to its preceding nodes along the same branch, ensuring that draft sequences from different branches do not interfere with each other. Specifically, the tree attention makes the nodes in the tree only “see” their preceding nodes, where each path from root to leaf can represent an individual draft sentence for LLMs due to the causality of LLMs. Therefore, the multiple potential draft sequences can be verified within SFP.

Problem formulation. For a query (or user) $q \in \mathcal{Q}$, the role of LLMs is to rerank N candidate items $\mathcal{D} = \{d_1, \dots, d_N\}$ (e.g., documents in IRs and products in RSs) based on their reasoning (see detailed prompts x^0 in Section 8.2 of the Appendix). Specifically, these candidate items can be noted by different single-token identifiers, e.g., $|A| \sim |Z|$, which can be used for ranking by LLMs (e.g., $|B| > |A| \dots > |E|$). In this paper, we aim to approximate the

ranking sequence π generated by auto-regressive decoding of the target LLM within SFP.

4 Methodology

We first reduce item ranking permutations to combinations based on the empirical observation (Section 4.1). Furthermore, we divide the full ranking into K sub-rankings, and introduce a Möbius inversion method with a hierarchical self-attention model to approximate the full ranking probability (Section 4.2).

4.1 Permutations $\rightarrow$ Combinations with Tree Attention

The tree attention can simultaneously validate multiple potential draft sequences, showing its potential to pre-verify multiple rankings that are required in the auto-regressive decoding within an SFP. Ideally, the lossless auto-regression decoding needs to pre-verify all permutations of subsets of given item candidates, i.e., $\mathcal{P} = \bigcup_{\mathcal{D}' \subseteq \mathcal{D}} \text{Perm}(\mathcal{D}')$, whose cardinality is $\sum_{m=0}^N m! \binom{N}{m}$ in order of growth of N . Although these permutations can be organized into a trie where different orders share common prefixes, the trie still expands to $N!$ nodes in total, and this factorial growth makes the SFP intractable.

To mitigate this, we explore relaxing permutations to combinations, i.e., approximating the ordered prefixes with unordered prefixes for auto-regression decoding. To validate this hypothesis, we conduct an experiment to measure the performance degradation when moving from auto-regressive decoding to different strategies. Our experimental setup is as follows: from a pool of 20 candidates, we first identify the top- L candidates using auto-regressive decoding. We then evaluate three distinct strategies for ranking the remaining $20 - L$ candidates:

- **FIRST:** This strategy ranks candidates solely based on the logit of the first token.
- **Permutation:** This method ranks the remaining $20 - N$ candidates based on logits conditioned on the exact ordered prefix (one of permutations) of N candidates obtained by auto-regressive decoding.
- **Combination:** This method ranks the remaining $20 - N$ candidates based on logits conditioned on the shuffled set (one of combinations) of the same N candidates.

Empirical results in Figure 2 (B) reveal that, while the combination strategy performs worse than the permutation strategy, it substantially outperforms FIRST. This finding suggests that the LLM's next-item generation is relatively insensitive to the exact ordering of previously ranked items, thereby justifying the relaxation from permutations to combinations in terms of effectiveness. Formally, at the t -step decoding, we estimate the probability of LLMs for selecting the next item conditioning on an unordered subset instead of a ranked sequence, i.e.,

$$i_{(t)} = \max_{i_{(t)}} \mathbb{P}(i_{(t)} \in \mathcal{D} / I_t | \text{Set}(I_t))$$

where I_t is an ordered ranking list (e.g., $[C, B, A, D]$) and $\text{Set}(I_t)$ denotes the unordered set of I_t (e.g., $[A, B, C, D]$). Therefore, we only need to pre-verify all combinations of subsets of given item candidates, i.e., $C = 2^{\mathcal{D}} := \{\mathcal{D}' | \mathcal{D}' \subseteq \mathcal{D}\}$.

Theoretically, such relaxation reduces the number of trie nodes to 2^N to verify C , which can be proved by the following theorem.

THEOREM 1 (LOWER BOUND OF COMBINATION-TRIE COMPLEXITY). *Let $\mathcal{D}$ be a candidate set with $|\mathcal{D}| = N$. When representing all possible subsets $S \subseteq \mathcal{D}$ in a prefix tree (trie) such that every subset corresponds to a unique root-to-node path, the number of nodes in the trie satisfies $|V| \geq 2^N$. Moreover, this lower bound is tight: there exists a construction of the trie with exactly 2^N nodes. (See proof in the appendix.)*

The transition from permutations ($N!$) to combinations (2^N) does not eliminate exponential complexity, but it reduces the search space from factorial to exponential scale, which is asymptotically much smaller (e.g., $20! \approx 2.4 \times 10^{18}$ versus $2^{20} \approx 10^6$).

4.2 Full Ranking $\rightarrow$ Sub-rankings with Möbius Inversion

Although utilizing combinations significantly prunes the verification space, the complexity of 2^N still remains intractable for an SFP, especially when the number of candidates N is large. To address this, we further reduce the complexity by randomly partitioning the candidate set $\mathcal{D}$ into K disjoint subsets with equal size, i.e., $\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \dots \cup \mathcal{D}_K$, reducing the size of the verification space from 2^N to $K \cdot 2^{\lceil N/K \rceil}$ and making it computationally feasible. However, a significant challenge arises from this partitioning: Some rankings are located in different subsets (e.g., $S \cap \mathcal{D}_i \neq \emptyset$ and $S \cap \mathcal{D}_j \neq \emptyset$), and naively aggregating sub-ranking probabilities in these subsets may lead to inaccurate estimation, i.e.,

$$\mathbb{P}(i | \cup_k S_k) \neq \sum_{k=1}^K \mathbb{P}(i | S_k),$$

where $S_k \subseteq \mathcal{D}_k$ denote the unsorted candidate items. For instance, the probability of choosing the next item given previously selected items is not a simple combination of their conditional probabilities, i.e., $\mathbb{P}(\cdot | A, B, C, D) \neq \mathbb{P}(\cdot | A, B) + \mathbb{P}(\cdot | C, D)$. Naively aggregating probabilities from sub-rankings ignores crucial cross-subset dependencies, biasing the final listwise probability.

Fortunately, with the pre-verification of all combinations in form of complete lattice ($\subseteq, 2^{\mathcal{D}_i}$), we can effectively model the individual effects of different selected items based on Möbius Inversion, showing potential to accurately approximate the listwise probability. Specifically, the core idea of Möbius Inversion is to decompose

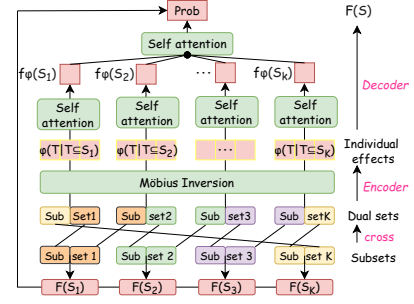


Figure 3: Möbius Inversion-based hierarchical self-attention model.

the observed, overall effects of item sets into their "pure" interaction components. Our tree-attention mechanism allows us to simultaneously verify numerous ranking prefixes in the form of a complete lattice within SFP. Therefore, it can seamlessly conduct reverse-engineering of these observations and isolate the marginal contribution of each interaction, contributing to accurately approximating the listwise probability.

4.2.1 Decomposing Probabilities with Möbius Inversion. In general, Möbius inversion works in the broader context of locally finite partially ordered sets (posets), i.e., $(P, \leq)$, where locally finite, meaning every interval $[x, y] = \{z \in P : x \leq z \leq y\}$ contains only finitely many elements. The general Möbius Inversion can be formulated as the reverse engineering between two functions $\phi, g : P \rightarrow A$, where A is an abelian group or ring. For any $g(x) = \sum_{y \leq x} f(y)$, there exists coefficient $\mu(y, x)$ that satisfies $f(x) = \sum_{y \leq x} \mu(y, x)g(y)$. If we implement posets $(P, \leq)$ as lattice $(2^{\mathcal{D}}, \subseteq)$, we can easily have $\mu(S, T) = (-1)^{|S-T|}$ if $T \subseteq S$.

In this paper, for each subset $\mathcal{D}_i$, we denote $F(S_i) = \mathbb{P}(\mathcal{D} | S_i) \in \mathbb{R}^N$ as the probability of the next token given that the items in set $S_i \subseteq \mathcal{D}_i$ have already been ranked. Based on Möbius Inversion, each probability $F(S_i)$ can be perfectly reconstructed by summing the pure interactions of all its subsets:

$$F(S_i) = \sum_{T \subseteq S_i} \phi(T) \quad (1)$$

where $\phi(T)$ isolates the marginal interaction effect of the items T , which can be formulated as

$$\phi(T) = \sum_{S \subseteq T} (-1)^{|T-S|} F(S) \quad (2)$$

where all cases can be obtained within SFP based on the prefix tree, i.e., $\{F(S) | S \in 2^{\mathcal{D}_i}\}$, we can estimate all marginal interaction effects, i.e., $\{\phi(S) | S \in 2^{\mathcal{D}_i}\}$, contributing to accurately approximating the listwise probability $F(S)$.

4.2.2 Approximation via hierarchical self-attention. Our goal is to accurately estimate $F(S)$ for any S during the decoding process, including first-order and high-order interactions of K subsets $\mathcal{D}_1, \dots, \mathcal{D}_K$, i.e.,

$$F(S) = F(\emptyset) + \underbrace{\sum_{i=1}^K \sum_{\emptyset \neq T_i \subseteq S_i} \phi(T_i)}_{\text{Within-group effects}} + \underbrace{\sum_{i=1}^K \sum_{\substack{\emptyset \neq T_i \subseteq S_i \\ \emptyset \neq T'_i \subseteq \mathcal{D} - \cup_{j \leq i} S_j}} \phi(T_i \cup T'_i)}_{\text{Cross-group (high-order) effects}}$$

where $S_i = S \cap \mathcal{D}_i$. To model the within-group effects, it can be formulated naively by aggregating only 1st-order effects according to the Möbius inversion, i.e.,

$$\sum_{i=1}^K \sum_{\emptyset \neq T_i \subseteq S_i} \phi(T_i) = \sum_{i=1}^K [F(S_i) - F(\emptyset)]; \quad (3)$$

To model the cross-group terms, we propose a hierarchical self-attention model to approximate their aggregation based on dual subsets, as computing all cross-group terms requires evaluating $F(S)$ for all $S \in 2^{\mathcal{D}}$. Specifically, we aim to construct the dual subsets that contain partial cross-set information, approximating the overall cross-subset contributions for full ranking decoding. To probe cross-set interactions, we adopt a partition-and-recombination strategy that repeatedly generates a new set by taking half of the items from a subset $T_i \subseteq S_i$ and the other half from a different subset $T_j \subseteq \mathcal{D} - \cup_{j \leq i} S_j$ without replacement. Therefore, the dual subsets $(\mathcal{D}'_1, \dots, \mathcal{D}'_K)$ satisfies item completeness, i.e., $\cup_i \mathcal{D}'_i = \cup_i \mathcal{D}_i$, and contain cross-set information, i.e., $\forall \mathcal{D}'_i \exists m \neq n$ with $\mathcal{D}'_i \cap \mathcal{D}_n \neq \emptyset$ and $\mathcal{D}'_i \cap \mathcal{D}_m \neq \emptyset$. Then, we design a hierarchical self-attention method to model the second-order and higher-order effects among cross-subsets. For the first layer, we adopt the self-attention mechanism on individual dual subsets, which contains cross-information between two different subsets for the second-order effect:

$$\begin{aligned} f_\phi(S_i) &= \sum_{S \subseteq S_i} \left[\phi(S) + \sum_{T \subseteq S_i} \alpha(S, T) \cdot \phi(T) \right] \in \mathbb{R}^N \\ \alpha(S, T) &= \frac{(\mathbf{W}_q \cdot \phi(S))^\top \cdot (\mathbf{W}_k \cdot \phi(T)) / \sqrt{N}}{\sum_{T' \subseteq S_i} (\mathbf{W}_q \cdot \phi(S))^\top \cdot (\mathbf{W}_k \cdot \phi(T')) / \sqrt{N}} \in \mathbb{R} \\ \phi(T) &= \sum_{S \subseteq T} (-1)^{|T-S|} F(S) \in \mathbb{R}^N \end{aligned}$$

where $\mathbf{W}_q \in \mathbb{R}$ and $\mathbf{W}_k$ denotes the projection matrix for query and key. For the second layer, we adopt the self-attention mechanism to model the interaction among dual subsets for capturing the higher-order effect, approximating the full ranking decoding $\hat{F}(S)$:

$$\begin{aligned} \hat{F}(S) &= \sum_{i=1}^K \left[f_\phi(S_i) + \sum_{j=1}^K \beta(i, j) \cdot f_\phi(S_j) \right] \in \mathbb{R}^N \\ \beta(i, j) &= \frac{(\mathbf{W}_q \cdot f_\phi(S_i))^\top \cdot (\mathbf{W}_k \cdot f_\phi(S_j)) / \sqrt{N}}{\sum_{j'=1}^K (\mathbf{W}_q \cdot f_\phi(S_i))^\top \cdot (\mathbf{W}_k \cdot f_\phi(S_{j'})) / \sqrt{N}} \in \mathbb{R} \end{aligned}$$

Möbius-based encoder-decoder. In the encoding phase, the Möbius transform on the Boolean lattice factors set-conditioned probabilities into marginal subset effects (i.e., $\{F(T)|T \in \cup_i 2^{\mathcal{D}_i}\} \rightarrow \{\phi(T)|T \in \cup_i 2^{\mathcal{D}_i}\}$). In the decoding phase, a hierarchical self-attention performs inversely, recombining these effects to recover the listwise probability (i.e., $\{\phi(T)|T \in \cup_i 2^{\mathcal{D}_i}\} \rightarrow \hat{F}(S)$).

4.2.3 Training and inference for auto-regression decoding. To align the training and inference phases of auto-regression decoding, we propose an auto-regressive training strategy for the Möbius Inversion model. Specifically, we start with an set $\mathcal{I}_0 = [x^0]$ only containing prompt x^0 . To simulate the auto-regression decoding, at the t -th step, we select the next item conditioning on an unordered subset, i.e.,

$$\mathcal{I}_t = \mathcal{I}_{t-1}.add(i_{(t)}), \quad i_{(t)} = \max_i \hat{F}_i(\mathcal{I}_{t-1}), \quad (4)$$

where $\hat{F}_i(\cdot)$ denotes the score of i -th item. To align our model with auto-regression decoding of target LLMs, we adopt a multi-classification cross-entropy loss to bridge between the generation of our model and target LLMs, i.e.,

$$\mathcal{L} = -\frac{1}{N^2} \sum_{t=0}^N \sum_{i=0}^N y_{ti} \cdot \log(\sigma(\hat{F}_i(\mathcal{I}_{t-1}))) + (1 - y_{ti}) \cdot \log(\sigma(1 - \hat{F}_i(\mathcal{I}_{t-1})))$$

where y_{ti} denotes the indicator that prioritizes the top-ranked items by target LLMs but not selected by ours until step t , i.e., $y_{ti} = 1$ if item $d_i \in \text{Top}(\pi - \mathcal{I}_{t-1}, \lambda)$ else 0. λ denotes the relaxation number for next-token selection. π is the ranking of auto-regressive decoding by the target LLM.

Inference and time complexity analysis. For the inference phase, we first pre-verify the subsets and dual subsets within SFP, and then “regressively decode” based on the Möbius Inversion model according to Equation 4 without multiple forward passes of LLMs. The primary advantage of our method lies in its communication efficiency. *Standard Auto-regressive Decoding* requires N sequential round trips to the LLM to rank N items. This process suffers from high communication between HBM and on-chip, making it slow and inefficient. The number of interactions is $O(N)$. *Our Method* and the STD method collapse the entire interaction into a single round trip. A large request containing all combinations is sent once, and a large response is received once. This reduces the number of interactions to $O(1)$. By eliminating $N - 1$ sequential calls, our approach dramatically reduces the communication cost. For computational cost, our method is inevitably larger than the STD method with an extra $2K \cdot 2^{\lceil N/K \rceil}$ tokens, but our method shows huge accuracy improvement. The analysis of practical wall-clock inference time on different methods is detailed in the experiments.

5 Experiments

In this section, we aim to validate the effectiveness of the proposed method ONE-PASS. Specifically, we conduct extensive experiments in both IR and RS tasks to study the following research questions: **RQ1:** Whether the proposed ONE-PASS outperforms state-of-the-art methods? **RQ2:** Whether the proposed ONE-PASS benefits from the proposed module, such as combination simplification, Möbius inversion, and the hierarchical self-attention model? **RQ3:** How does the proposed ONE-PASS generalize across different datasets, tasks, and backbones? **RQ4:** How do hyper-parameters influence the performance of the proposed ONE-PASS?

5.1 Experimental Setup

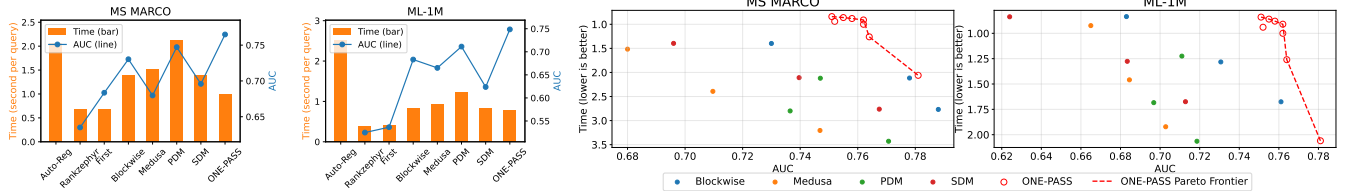
5.1.1 Tasks and Datasets. To comprehensively assess the efficacy of our proposed method, ONE-PASS, we conduct experiments on two representative ranking tasks: Information Retrieval (IR) and Recommender Systems (RS) [17]. In both tasks, the fundamental objective is to rank a set of candidate passages (e.g., documents in IR, items in RS) with a given query [13, 24, 27]. For the IR task, we employ two distinct datasets, namely MS MARCO and Quora. The MS MARCO dataset is prepared by [24]¹, and we filter for instances that contain exactly 20 candidate passages for reranking, which yields a final dataset of 9,978 examples. The Quora dataset² is sourced from Quora.com, and we randomly sample 5,000 queries that are

¹https://huggingface.co/datasets/rryisthebest/rank_zephyr_training_data_alpha

²<https://public.ukp.informatik.tu-darmstadt.de/thakur/BEIR/datasets/quora.zip>

Table 2: Performance of different methods on IR tasks and RS tasks with different LLM backbones. * indicates statistically significant improvement of ONE-PASS to baselines on t-test ($p < 0.05$).

Datasets	Backbones	Llama-3.2-3B-Instruct					Qwen2.5-7B-Instruct				
		AUC	SP	KD↓	Recall@10	NDCG@10	AUC	SP	KD↓	Recall@10	NDCG@10
MS MARCO	Rankzephyr	0.6350	0.3653	69.34	0.6180	0.6706	0.6758	0.4709	61.60	0.6872	0.7460
	First	0.6835	0.4823	60.14	0.6708	0.7143	0.5837	0.2057	79.09	0.5862	0.6136
	Blockwise	0.7303	0.5906	51.24	0.7194	0.7718	0.7704	0.6805	43.63	0.7718	0.8267
	Medusa	0.6797	0.4742	60.85	0.6524	0.7237	0.7365	0.6094	50.06	0.7182	0.7901
	PDM	0.7472	0.6063	48.04	0.7346	0.8029	0.7412	0.5858	49.16	0.7230	0.8000
	SDM	0.6959	0.5131	57.79	0.6698	0.7387	0.7427	0.6215	48.89	0.7374	0.8014
	ONE-PASS	0.7649*	0.6555*	44.66*	0.7392	0.7998	0.7871*	0.7067*	40.46*	0.7742	0.8346*
Quora	Rankzephyr	0.4849	-0.0360	154.52	0.4042	0.4442	0.6501	0.4143	104.96	0.6390	0.7161
	First	0.6627	0.4463	101.19	0.6268	0.7031	0.5551	0.1616	133.48	0.5012	0.5569
	Blockwise	0.7060	0.5391	88.20	0.6778	0.7486	0.7205	0.5796	83.85	0.7324	0.7951
	Medusa	0.6672	0.4490	99.84	0.6032	0.6909	0.7061	0.5404	88.18	0.6644	0.7518
	PDM	0.7334	0.5847	79.97	0.7008	0.7781	0.6861	0.4842	94.16	0.6528	0.7463
	SDM	0.6578	0.4269	102.66	0.6244	0.7078	0.7035	0.5391	88.94	0.6938	0.7713
	ONE-PASS	0.7414*	0.6110*	77.57*	0.7088	0.7795	0.7379*	0.6119*	78.63*	0.7296	0.8021*
ML-1M	Rankzephyr	0.5249	0.0752	142.53	0.4602	0.5184	0.5456	0.1289	136.31	0.4580	0.4821
	First	0.5365	0.1084	139.06	0.4628	0.5367	0.5435	0.1245	136.94	0.4620	0.4839
	Blockwise	0.6832	0.4565	95.03	0.6270	0.6995	0.7256	0.5927	82.33	0.6986	0.7637
	Medusa	0.6651	0.4113	100.46	0.5972	0.6682	0.6757	0.4735	97.29	0.6274	0.6995
	PDM	0.7113	0.5433	86.60	0.6314	0.7135	0.6997	0.5088	90.10	0.6704	0.7519
	SDM	0.6237	0.3397	112.89	0.5776	0.6506	0.7126	0.5618	86.21	0.6802	0.7451
	ONE-PASS	0.7486*	0.6189*	75.42*	0.6830*	0.7484*	0.7485*	0.6316*	75.45*	0.7162*	0.7828*
Amazon-Games	Rankzephyr	0.4691	-0.0694	159.28	0.3830	0.4486	0.5977	0.2829	120.69	0.5680	0.6441
	First	0.5822	0.2293	125.33	0.5176	0.5822	0.5465	0.1428	136.05	0.4820	0.5482
	Blockwise	0.6939	0.4834	91.82	0.6536	0.7173	0.7391	0.6163	78.28	0.7230	0.7820
	Medusa	0.6737	0.4201	97.88	0.6276	0.6899	0.7041	0.5430	88.76	0.6474	0.7298
	PDM	0.7198	0.5639	84.06	0.6444	0.7205	0.7286	0.5761	81.41	0.6984	0.7764
	SDM	0.6145	0.3167	115.65	0.5626	0.6452	0.7095	0.5472	87.14	0.6770	0.7460
	ONE-PASS	0.7383*	0.5981*	78.50*	0.6498	0.7244*	0.7794*	0.6940*	66.17*	0.7484*	0.8094*

**Figure 4: (A) The statistics of practical wall-clock inference time for different models, where the orange bars indicate the time cost (sec/query) and the blue lines indicate approximation accuracy measured by AUC. (B) Accuracy–Latency Pareto Frontier analysis.**

paired with 25 candidate passages previously retrieved using the BM25 model. For the RS task, which focuses on recommending preferred items to users, we adopt two widely used datasets ML-1M and Amazon-Game. The ML-1M dataset³ contains 1,000,000 interactions from 6,040 users, while the Amazon-Game dataset⁴ includes 231,780 interactions from 9,512 users. For every user in both datasets, we construct a reranking list of 25 candidate items, consisting of 8 items the user has positively engaged with and 17 items with which the user has not interacted.

5.1.2 Backbone models and baselines. In this paper, we select two representative LLM backbones with different sizes and series, namely Llama-3.2-3B-Instruct [11] and Qwen2.5-7B-Instruct [34]. For baseline methods, we take the following state-of-the-art methods with STD strategy, i.e., Rankzephyr [23] (2023), FIRST [24] (2024), and SD-based methods, i.e., Speculative Decoding Method (SDM,

2023) [16], Blockwise [26] (2018), Medusa [2] (2024), and Parallel Decoding Method (PDM) [25] (2023). For fair comparison, we directly rank the unaccepted items based on the logits of the last pass forward when SD-based methods reach a comparable latency to ours (usually 2-times pass forward of LLMs).

5.1.3 Evaluation Protocol and Metrics. For all datasets, we reserve the last 500 queries are reserved for the validation set, and the 500 queries immediately preceding them are designated as the test set. All remaining data points are utilized for training. During the evaluation, we measure performance using two categories of metrics [20]. To evaluate the similarity between the entire predicted ranking and the target ranking, we use the AUC, Spearman’s Rho (SR), and Kemeny Distance (KD). To assess the accuracy of the top-ranked items, we use Recall@10 and NDCG@10. Since our goal is to approximate the ranking produced by the target LLM’s auto-regressive generation, we consider the top 10 items from that output as the ground truth for our Top-k evaluations. Experimental results are reported as the average over five runs.

³<https://grouplens.org/datasets/movielens/>⁴<https://jmcauley.ucsd.edu/data/amazon/>

Table 3: Ablation studies on IR and RS tasks on Llama-3.2-3B-Instruct.

Datasets	Methods	AUC	SP	KD	R@10	N@10
MS MARCO	STD	0.6502	0.4002	66.47	0.6300	0.6940
	Permutation	0.7351	0.6027	50.32	0.7044	0.7805
	Naïve add	0.7448	0.6179	48.48	0.7080	0.7772
	wo Möbius-1	0.7479	0.6251	47.90	0.7128	0.7810
	wo Möbius-2	0.7486	0.6263	47.76	0.7140	0.7814
	wo HAS	0.7475	0.6243	47.97	0.7124	0.7807
	ONE-PASS	0.7649	0.6555	44.66	0.7392	0.7998
Quora	STD	0.6189	0.3323	114.3	0.5802	0.6648
	Permutation	0.6974	0.5203	90.79	0.6628	0.7531
	Naïve add	0.7246	0.5758	82.62	0.6826	0.7598
	wo Möbius-1	0.7262	0.5780	82.13	0.6834	0.7603
	wo Möbius-2	0.7263	0.5778	82.12	0.6852	0.7598
	wo HAS	0.7262	0.5780	82.14	0.6834	0.7603
	ONE-PASS	0.7414	0.6110	77.57	0.7088	0.7795
ML-1M	STD	0.5336	0.1049	139.9	0.4712	0.5540
	Permutation	0.6858	0.4952	94.25	0.6566	0.7381
	Naïve add	0.7127	0.5490	86.17	0.6482	0.7251
	wo Möbius-1	0.7108	0.5437	86.76	0.6396	0.7169
	wo Möbius-2	0.7089	0.5404	87.32	0.6396	0.7180
	wo HAS	0.7063	0.5339	88.10	0.6376	0.7156
	ONE-PASS	0.7486	0.6189	75.42	0.6830	0.7484
Amazon-Games	STD	0.5437	0.1324	136.8	0.4780	0.5654
	Permutation	0.6682	0.4464	99.53	0.6278	0.7163
	Naïve add	0.6900	0.4975	93.00	0.6100	0.6960
	wo Möbius-1	0.6898	0.4981	93.04	0.6062	0.6930
	wo Möbius-2	0.6877	0.4940	93.68	0.6068	0.6940
	wo HAS	0.6870	0.4911	93.88	0.6040	0.6915
	ONE-PASS	0.7383	0.5981	78.49	0.6498	0.7244

5.1.4 Implementation Details. For implementation details, we set the learning rate to $1e-3$ using the Adam optimizer and a batch size of 16 for all methods. For our method, we divide the full ranking into $K = 5$ sub-rankings for IR tasks and $K = 6$ sub-rankings for RS tasks. For baseline methods, we use the parameter settings provided by the authors when available; otherwise, we tune them for optimal performance under the constrained budget and fair computational complexity.

5.2 Model Comparison (RQ1).

We examine whether the proposed ONE-PASS framework surpasses state-of-the-art methods in both IR and RS tasks. Table 2 reports the performance of representative approaches, including STD-based methods and several speculative decoding variants, across IR and RS tasks under different LLM backbones. The experimental findings yield the following observations: First, ONE-PASS consistently achieves superior performance over all baselines in most cases, underscoring its effectiveness. Its relative gains, however, are less pronounced on top- n metrics (i.e., Recall@10 and NDCG@10). This limitation is primarily attributable to speculative decoding techniques are tailored for precise decoding and thus exhibit strong accuracy in predicting top-ranked items. Second, although STD methods (Rankzephyr and FIRST) are capable of decoding all items within a single forward pass using only the original prompt, they underperform compared to other approaches due to pronounced degradation in tail rankings. Third, the FIRST method surpasses Rankzephyr under fine-tuned LLMs, a result that can be ascribed to its adoption of RankNet loss, which emphasizes ranking accuracy for highly relevant passages.

Furthermore, Figure 4 (A) illustrates the practical wall-clock inference time across models. First, ONE-PASS attains markedly higher accuracy (measured by AUC) relative to other SD-based models while maintaining comparable latency, highlighting its practical utility for efficient and accurate inference. Second, in comparison with auto-regressive decoding enhanced by KV caching, ONE-PASS achieves an acceleration of approximately $2\times \sim 5\times$. Third, although FIRST and Rankzephyr exhibit the fastest decoding speed within SPF with only the original prompt, this advantage comes at the expense of substantial accuracy loss. In addition, Figure 4 (B) illustrates the practical wall-clock inference time across models under different time budgets (i.e., different K for ONE-PASS and various steps for speculative decoding methods). It shows that ONE-PASS achieves the Pareto frontier among these methods. In conclusion, ONE-PASS effectively accelerates LLM decoding without compromising accuracy, demonstrating its efficiency and superiority.

5.3 Ablation Studies (RQ2).

To validate the contributions of each critical component in our ONE-PASS method, we conducted comprehensive ablation studies. Specifically, we examined the following variants:

STD (Single Token Decoding): A baseline variant that relies solely on single-token decoding on the Villina backbone without fine-tuning. **Permutation**: Instead of using combinations, it directly pre-verify all possible prefix permutations. As it is also an SPF method, we pre-verify permutations of the first 3 prefixes ($K \cdot (K - 1) \cdot (K - 2)$ tokens, which are more than ours), and choose the ground truth one to rerank for the remaining items. **Naïve add**: We naively aggregate sub-ranking probabilities for estimation, i.e., $\mathbb{P}(i| \cup_k S_k) = \sum_{k=1}^K \mathbb{P}(i|S_k)$. **wo Möbius**: We remove the Möbius Inversion in ONE-PASS. As the Möbius Inversion is highly coupled into ONE-PASS, we adopt two strategies for comprehensive studies. First, we only adopt the attention mechanism that learn to aggregate the sub-ranking as well as dual sub-ranking probabilities $\{\mathbb{P}(i|S_1) \cdots, \mathbb{P}(i|S_1)\} \cup \{\mathbb{P}(i|S'_1) \cdots, \mathbb{P}(i|S'_1)\}$, denoted as **wo Möbius-1**. Second, we replace the individual effect $\phi(T)$ with the coupled probability $F(T)$, denoted as **wo Möbius-2**. **wo HAS**: We remove the proposed hierarchical self-attention model. Instead, we directly aggregate the individual effects of each subset, i.e., $f_\phi(S_i) = \sum_{T \subseteq S_i} \phi(T)$.

Table 3 reports the detailed results of these ablation experiments on both IR and RS tasks using Llama-3.2-3B backbone. From these results, we draw the following observations: First, the variant **STD** shows the worst performance among all methods, underscoring the gap between the logits of the first token and the rankings produced by full auto-regressive decoding. This highlights the necessity of more expressive approximations. Second, the **Permutation** variant performs better than STD by explicitly verifying several prefix permutations, but the computational overhead grows quickly, making it perform worse than the combination-based methods within a limited token budget. This confirms the benefit of replacing permutations with combinations to balance efficiency and effectiveness. Third, the **Naïve add** baseline improves upon Permutation but still lags behind the proposed ONE-PASS, demonstrating that simply aggregating sub-ranking probabilities cannot capture the interdependencies across subsets, leading to inaccurate estimation in

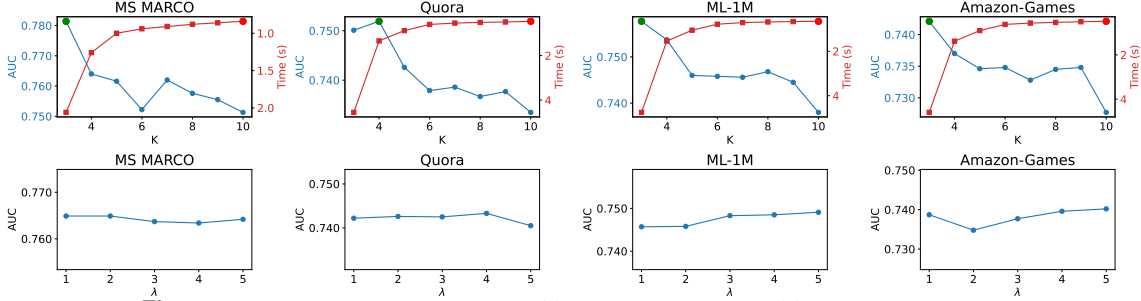


Figure 5: Hyper-parameters analysis on (a) subset number K and (b) relaxation number λ .

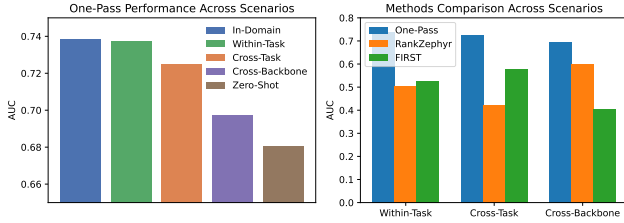


Figure 6: Generalization and robustness analysis.

listwise ranking. This validates the need for a more principled aggregation scheme. Fourth, the **wo Möbius-1/2** variants also show degradation even with some cross-subsets information provided by dual sub-ranking. Specifically, the **wo Möbius-1** variant adopts an attention mechanism that learn to aggregate the sub-ranking (as well as dual sub-ranking) probabilities. However, it fails to capture the individual effect of different subsets, making it hard to model the higher-order effects among cross-subsets. Although the **wo Möbius-2** variant attempts to explicitly model conditional probabilities for all subsets, these probabilities are essentially the decoupled effects of different subsets, making it hard to effectively approximate full ranking probabilities. Finally, the **wo HAS** variant also underperforms compared to ONE-PASS, showing that hierarchical self-attention is effective in modeling high-order cross-subset interactions. In fact, ONE-PASS will degrade into **Naïve add** of dual sub-ranking without a hierarchical self-attention model.

5.4 Generalization Analysis (RQ3)

To evaluate the practical utility of ONE-PASS, we conduct extensive Out-of-Distribution (OOD) evaluations on the Amazon-Games dataset with the Llama-3.2-3B backbone. We examine four levels of distribution shift to assess how our model generalizes compared to in-domain baselines: **Within-Task** is trained on a different dataset within the recommendation task (i.e., ML-1M). **Cross-Task** (IR $\rightarrow$ RS) is trained on an IR dataset (i.e., Quora) to evaluate cross-domain transferability. **Cross-Backbone** is trained on Qwen2.5-7B while being tested on Llama-3.2-3B to measure architecture sensitivity. The Experiment results are shown as follows:

We highlight that ours performs better than all in-domain-trained baselines in Within- and Cross-Task scenarios, validating its generalization. Furthermore, generalization scales with the similarity between training and testing environments. For instance, the Within-Task scenario achieves the highest performance, while the Cross-Task scenario shows a slight drop. While baselines like Rankzephyr and FIRST exhibit severe degradation in OOD settings, ONE-PASS

remains effective because it explicitly approximates the LLM’s autoregressive decoding process.

5.5 Hyper-parameter Analysis (RQ4).

In this part, we investigate how hyper-parameters influence the effectiveness and efficiency of the proposed ONE-PASS framework, focusing on the number of subsets K and the relaxation parameter λ . Intuitively, increasing the number of subsets K reduces the token cost required for parallel ranking verification, but simultaneously makes probability estimation more difficult. As depicted in Figure 6, when K increases, the inference time consistently decreases across all datasets. However, this efficiency gain comes at the cost of ranking quality. The AUC curves exhibit a gradual decline as K increases, with the best performance typically achieved around $K = 3$ or $K = 4$, after which the AUC starts to decline steadily. This reveals a clear trade-off between efficiency and effectiveness. In real-world scenarios, the K can be flexibly selected based on the needs of ranking systems. For the relaxation parameter λ , it is observed that the performance of ONE-PASS remains stable (slightly improved in most cases) with the increase of λ . We suggest implementing λ using a grid search strategy.

6 Conclusion

In this paper, we introduce an SFP-based method to pre-verify multiple sub-rankings using tree attention, approximating autoregressive decoding by relevant sub-rankings at each step. To alleviate the intensive tokens within SFP, we reduce item ranking permutations to combinations and divide the full ranking into sub-rankings, making it efficient for listwise reranking. In addition, we employ the Möbius inversion to decouple into individual effects of each subset, accurately approximating the probability of full ranking. The proposed ONE-PASS is a backbone tuning-free method, which can be utilized as a plugin to accelerate the inference for LLM-based reranking. Experiments on IR and RS tasks demonstrate that our method significantly reduces latency while maintaining high-quality performance.

7 Acknowledgement

This research is supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG3-RP-2022-031), and A*STAR under its MTC Programmatic (Award M23L9b0052). This research is supported in part by the Singapore MOE AcRF Tier 1 funding (RG16/25).

References

- [1] Edward A Bender and Jay R Goldman. 1975. On the applications of Möbius inversion in combinatorial analysis. *The American Mathematical Monthly* 82, 8 (1975), 789–803.
- [2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. MEDUSA: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*. 5209–5235.
- [3] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [4] Henry H Crapo. 1969. Möbius inversion in lattices. *Archiv der Mathematik* 19, 6 (1969), 595–607.
- [5] Cunxiao Du, Jing Jiang, Xu Yuanchen, Jiawei Wu, Sicheng Yu, Yongqi Li, Shenggui Li, Kai Xu, Liqiang Nie, Zhaopeng Tu, et al. 2024. GLIDE with a CAPE: a low-hassle method to accelerate speculative decoding. In *Proceedings of the 41st International Conference on Machine Learning*. 11704–11720.
- [6] Yingpeng Du, Hongzhi Liu, Hengshu Zhu, Yang Song, Zhi Zheng, and Zhonghai Wu. 2025. Quasi-Metric Learning for Bilateral Person-Job Fit. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [7] Yingpeng Du, Di Luo, Rui Yan, Xiaopei Wang, Hongzhi Liu, Hengshu Zhu, Yang Song, and Jie Zhang. 2024. Enhancing job recommendation through llm-based generative adversarial networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8363–8371.
- [8] Yingpeng Du, Zhu Sun, Ziyang Wang, Haoyan Chua, Jie Zhang, and Yew-Soon Ong. 2025. Active Large Language Model-based Knowledge Distillation for Session-based Recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 11607–11615.
- [9] Yingpeng Du, Tianjun Wei, Zhu Sun, and Jie Zhang. 2025. Reinforcement Speculative Decoding for Fast Ranking. *arXiv preprint arXiv:2505.20316* (2025).
- [10] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *International Conference on Machine Learning*. PMLR, 14060–14079.
- [11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [12] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Sophia Shao. 2023. Speed: Speculative pipelined execution for efficient decoding. *arXiv preprint arXiv:2310.12072* (2023).
- [13] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024. Large language models are zero-shot rankers for recommender systems. In *European Conference on Information Retrieval*. Springer, 364–381.
- [14] Sehoon Kim, Kartikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W Mahoney, Amir Gholami, and Kurt Keutzer. 2023. Speculative decoding with big little decoder. *Advances in Neural Information Processing Systems* 36 (2023), 39236–39256.
- [15] Wouter Kool, Herke Van Hoof, and Max Welling. 2020. Ancestral gumbel-top-k sampling for sampling without replacement. *Journal of Machine Learning Research* 21, 47 (2020), 1–36.
- [16] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
- [17] Hongzhi Liu, Yingpeng Du, and Zhonghai Wu. 2022. Generalized ambiguity decomposition for ranking ensemble learning. *Journal of Machine Learning Research* 23, 88 (2022), 1–36.
- [18] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2024. Once: Boosting content-based recommendation with both open-and closed-source large language models. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 452–461.
- [19] Chuan Meng, Negar Arabzadeh, Arian Askari, Mohammad Aliannejadi, and Maarten de Rijke. 2024. Ranked list truncation for large language model-based re-ranking. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 141–151.
- [20] Kevin P Murphy. 2012. *Machine learning: a probabilistic perspective*. MIT press.
- [21] Andrew Parry, Sean MacAvaney, and Debasis Ganguly. 2024. Top-down partitioning for efficient list-wise ranking. *arXiv preprint arXiv:2405.14589* (2024).
- [22] Ronak Pradeep, Sahel Sharifmoghadam, and Jimmy Lin. 2023. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088* (2023).
- [23] Ronak Pradeep, Sahel Sharifmoghadam, and Jimmy Lin. 2023. RankZephyr: Effective and Robust Zero-Shot Listwise Reranking is a Breeze! *arXiv preprint arXiv:2312.02724* (2023).
- [24] Revanth Gangi Reddy, JaeHyek Doo, Yifei Xu, Md Arafat Sultan, Deevya Swain, Avirup Sil, and Heng Ji. 2024. FIRST: Faster Improved Listwise Reranking with Single Token Decoding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 8642–8652.
- [25] Andrea Santilli, Silvio Severino, Emiliano Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, Emanuele Rodola, et al. 2023. Accelerating Transformer Inference for Translation via Parallel Decoding. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vol. 1. Association for Computational Linguistics, 12336–12355.
- [26] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems* 31 (2018).
- [27] Ziyang Wang, Yingpeng Du, Zhu Sun, Haoyan Chua, Kaidong Feng, Wenya Wang, and Jie Zhang. 2025. Re2llm: Reflective reinforcement large language model for session-based recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12827–12835.
- [28] Tianjun Wei, Huizhong Guo, Yingpeng Du, Zhu Sun, Huang Chen, Dongxia Wang, and Jie Zhang. 2025. Mirroring Users: Towards Building Preference-aligned User Simulator with User Feedback in Recommendation. *arXiv preprint arXiv:2508.18142* (2025).
- [29] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.
- [30] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 12–22.
- [31] Yunjia Xi, Hangyu Wang, Bo Chen, Jianghao Lin, Menghui Zhu, Weiwen Liu, Ruiming Tang, Zhewei Wei, Weinan Zhang, and Yong Yu. 2025. Efficiency unleashed: Inference acceleration for LLM-based recommender systems with speculative decoding. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1891–1901.
- [32] Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. 2023. Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 3909–3925.
- [33] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. In *ACL (Findings)*.
- [34] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- [35] Seongjun Yang, Gibbeum Lee, Jaewoong Cho, Dimitris Papailiopoulos, and Kangwook Lee. [n. d.]. Predictive Pipelined Decoding: A Compute-Latency Trade-off for Exact LLM Decoding. *Transactions on Machine Learning Research* [n. d.].
- [36] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024. Draft& Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11263–11282.
- [37] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* 1, 2 (2023).
- [38] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhao Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2023. Large language models for information retrieval: A survey. *arXiv preprint arXiv:2308.07107* (2023).

8 Appendix

8.1 Detailed Proof of Theorems

THEOREM 1 (LOWER BOUND OF COMBINATION-TRIE COMPLEXITY).
Let $\mathcal{D}$ be a candidate set with $|\mathcal{D}| = N$. When representing all possible subsets $S \subseteq \mathcal{D}$ in a prefix tree (trie) such that every subset corresponds to a unique root-to-node path, the number of nodes in the trie satisfies $|V| \geq 2^N$. Moreover, this lower bound is tight: there exists a construction of the trie with exactly 2^N nodes.

PROOF. Every distinct subset $S \subseteq \mathcal{D}$ must appear as a unique node to satisfy the prefix property. Since there are 2^N subsets, at least 2^N nodes are required. To show tightness, we fix an arbitrary total order on $\mathcal{D}$, e.g., $c_1 < c_2 < \dots < c_N$. For each subset $S = \{c_{i_1}, \dots, c_{i_k}\}$ with $i_1 < i_2 < \dots < i_k$, we can encode S as the increasing sequence $(c_{i_1}, c_{i_2}, \dots, c_{i_k})$ and insert this path into the trie. This construction produces exactly 2^N distinct nodes (each

corresponding to a unique subset, with the empty set as root) and $2^N - 1$ edges. Hence, the lower bound is attained. $\square$

8.2 Prompt for LLMs in IR and RS tasks

Prompt for LLMs in IR task.

System: You are RankLLM, an intelligent assistant that can rank passages based on their relevancy to the query.

User: I will provide you with 20 passages, each indicated by an identifier [?]. Rank the passages based on their relevance to the search query: *[can you use paints on leather]*.

|A| In order to remove these and allow the paint to bind, ... strip away the majority of the protective layers.

...

|T| Most solutions used to remove paint will also remove the pigment from your leather furniture... Dark drinks will need specific care methods.

Search Query: *[can you use paints on leather]*.

Rank the 20 passages above based on their relevance to the search query. All the passages should be included and listed using identifiers, in descending order of relevance. The output format should be formulated with $|?|>|?|$, e.g., $|C|>|B|>|A|>|D|$. Only respond with the ranking results, do not say any word or explain.

Prompt for LLMs in RS task.

System: You are a Recommender expert, an intelligent assistant that can rank candidate items (e.g., movies) by analyzing user history behaviors.

User: I will first provide you with user's historical behaviors (watching movies). The title of this user watched movie are *<Princess Bride, The (1987)>*, ..., *<Dick Tracy (1990)>*. Now, I will provide you with 25 movies, each indicated by an identifier [?]. Please rank these movies based on user preference and her historical behaviors:

|A| Honey, I Shrunk the Kids (1989)

|B| Back to the Future Part II (1989)

|C| Who Framed Roger Rabbit (1988)

|D| The Mask (1994)

...

|Y| Perez Family, The (1995)

Rank these 25 candidate movies above based on user preference and her historical behaviors. All the items should be included and listed using identifiers, in descending order of relevance. The output format should be formulated with $|?|>|?|$, e.g., $|C|>|B|>|A|>|D|$. Only respond with the ranking results, do not say any words or explain.